

第一次实验：基于 OpenMP 的多线程 Conv 算子实现

2023302121009 刘鼎琨

一、实验源码

1. 串行函数代码：

```
181 ~ void conv_serial(const std::vector<float>& image,  
182 ~     const std::vector<float>& kernel,  
183 ~     std::vector<float>& out,  
184 ~     const ConvShape& shape) {  
185 ~     for (int oh = 0; oh < shape.oh(); ++oh) {  
186 ~         for (int ow = 0; ow < shape.ow(); ++ow) {  
187 ~             float sum = 0.0f;  
188 ~             for (int c = 0; c < shape.c; ++c) {  
189 ~                 for (int kh = 0; kh < shape.kh; ++kh) {  
190 ~                     for (int kw = 0; kw < shape.kw; ++kw) {  
191 ~                         sum += image[image_index(shape, c, oh + kh, ow + kw)]  
192 ~                             * kernel[kernel_index(shape, c, kh, kw)];  
193 ~                     }  
194 ~                 }  
195 ~             }  
196 ~             out[out_index(shape, oh, ow)] = sum;  
197 ~         }  
198 ~     }  
199 ~ }
```

图 1 串行函数代码

2. 并行函数代码：

```
208 ~ void conv_openmp(const std::vector<float>& image,  
209 ~     const std::vector<float>& kernel,  
210 ~     std::vector<float>& out,  
211 ~     const ConvShape& shape) {  
212 ~     #pragma omp parallel for schedule(static)  
213 ~     for (int oh = 0; oh < shape.oh(); ++oh) {  
214 ~         for (int ow = 0; ow < shape.ow(); ++ow) {  
215 ~             float sum = 0.0f;  
216 ~             for (int c = 0; c < shape.c; ++c) {  
217 ~                 for (int kh = 0; kh < shape.kh; ++kh) {  
218 ~                     for (int kw = 0; kw < shape.kw; ++kw) {  
219 ~                         sum += image[image_index(shape, c, oh + kh, ow + kw)]  
220 ~                             * kernel[kernel_index(shape, c, kh, kw)];  
221 ~                     }  
222 ~                 }  
223 ~             }  
224 ~             out[out_index(shape, oh, ow)] = sum;  
225 ~         }  
226 ~     }  
227 ~ }
```

图 2 并行函数代码

二、运行结果

1. 执行终端指令：

```
D:\并行与分布式计算\课程实验> g++ -O2 -std=c++17 -fopenmp
conv_openmp_starter.cpp -o conv_openmp_starter.exe

D:\并行与分布式计算\课程实验> .\conv_openmp_starter.exe 1
```

2. 运行结果：

```
PS D:\大学\大三下\学习\并行与分布式计算\课程实验> .\conv_openmp_starter.exe 1
data:      conv_data.bin
image:     3x1024x1024
kernel:    3x3x3
output:     1022x1022
threads:    1
serial_max_diff: 0.000
serial_status: PASS
openmp_max_diff: 0.000
openmp_status: PASS
serial_openmp_max_diff: 0.000
serial_ms:  41.783
openmp_ms:  34.544
speedup:    1.210
checksum:   -521733.854
```

图 3 运行结果截图

三、加速比测试

核数 (Cores)	串行耗时 (serial_ms)	OpenMP 耗时 (openmp_ms)	加速比 (Speedup)
1	41.783	34.544	1.210
2	44.586	25.623	1.740
4	39.930	18.295	2.183
8	46.413	15.434	3.007
12	43.780	17.527	2.498
16	46.261	23.239	1.991

通过调整参数形成表 1 的加速比测试结果，将其绘制为图 4 的折线图：

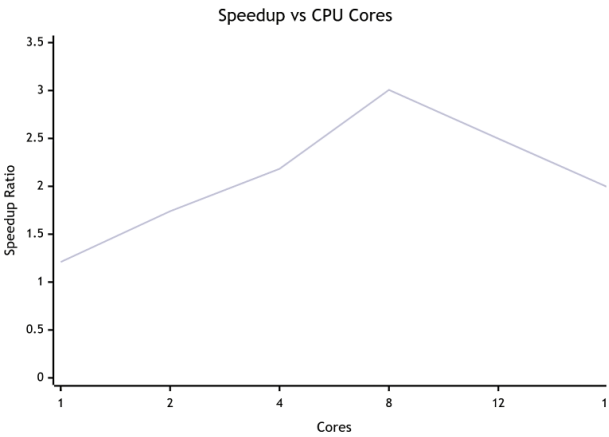


图 4 性能折线图

四、正确性截图

```
serial_max_diff: 0.000
serial_status:   PASS
openmp_max_diff: 0.000
openmp_status:   PASS
```

图 5 正确性验证截图

五、加速比变化解释

1. 性能上升阶段（1-8 核）

在这个阶段，加速比从 1.21 提升到了峰值 3.007。卷积操作具有很高的计算密度。当增加核心数时，OpenMP 将 oh 循环的任务分配到不同的处理器核心上，使得原本需要串行等待的计算任务可以同时进行，此时的并行收益大于开销。

2. 性能衰退阶段（12-16 核）

从 12 核开始，加速比从 3.007 跌至 1.991。当核心数过多时，OpenMP 用于管理线程、任务调度和同步的开销会显著增加。如果每个线程分配到的计算量太小，线程切换的时间甚至会超过实际计算的时间。同时，卷积是访存密集型任务。虽然有 16 个核心在算，但所有核心共享同一个内存通道。当 16 个线程同时向内存请求数据时，会发生冲突问题

3. 异常现象分析：为什么 1 核加速比是 1.21？

1 核 OpenMP 的耗时（34.544ms）比纯串行（41.783ms）还要快。这通常是因为你在编译时开启了 -O2 优化。OpenMP 编译器在处理 `#pragma omp parallel for` 时，可能会触发一些额外的循环转换或矢量化优化，使得单线程版本也获得了比原始串行代码更好的性能。